

Informe Tarea 1

Pablo Donoso Salvo, pdonosos@usm.cl

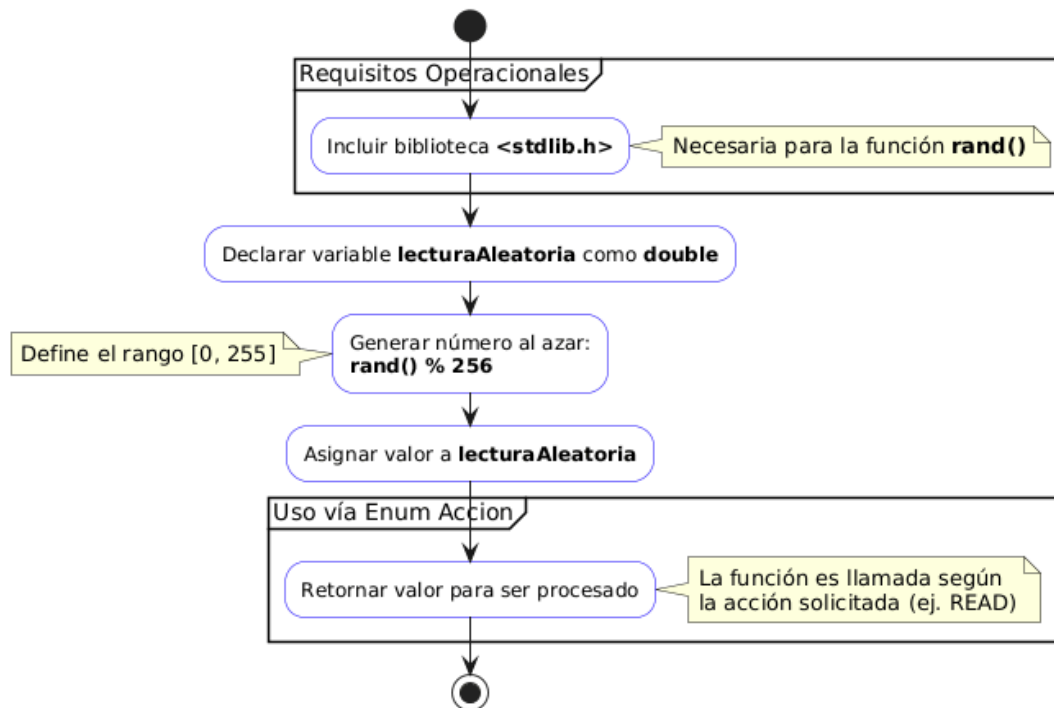
Isidora Vanerio Ceballos, ivanerio@usm.cl

1. Parte 1

1.1. Diseño de la función lecturaSensor()

Uno de los principales requisitos es la inclusión de la biblioteca `<stdlib.h>`, además es necesario declarar la variable `lecturaAleatoria` como un `double` y declararla con un `rand()` seguido de `%256` para definir que se genere un número al azar entre 0 y 255. Al utilizar el enum `Accion` lo que hacemos es pedir un numero al azar a esa función cuantas veces deseemos para poder utilizarlo como un simulador de sensores.

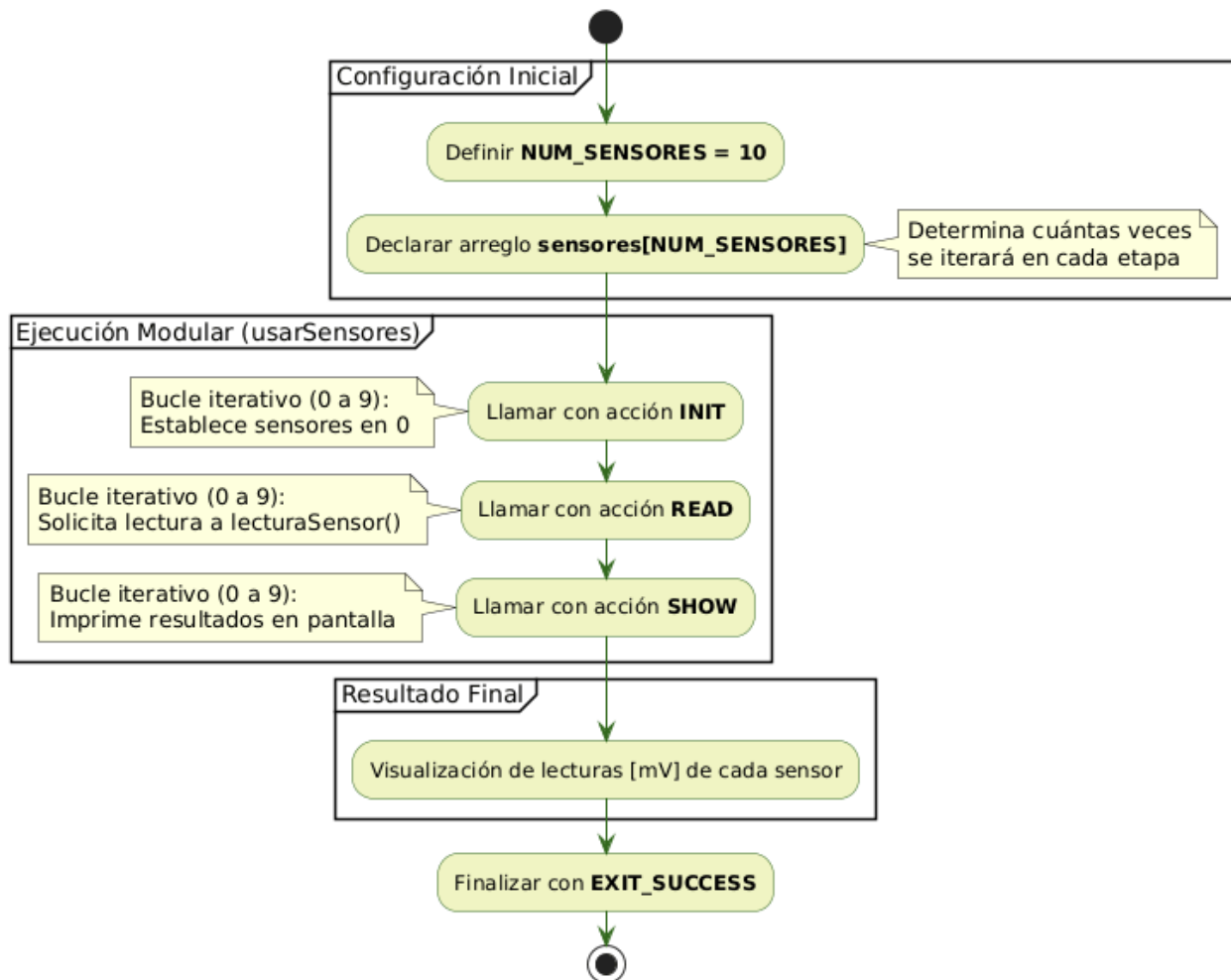
Función: lecturaSensor() (Simulador de Sensores)



1.2. Diseño completo de la nueva función main()

Con los cambios integrados `Main` inicia declarando la variable `sensores` que declara la veces que se va a iterar el código, luego se usa la función `usarSensores` según cada apartado del enum (`INT`, `READ`, `SHOW`), una vez que pasa por las 3 partes `n` veces (según `NUM_SENsores`) lo que se verá en pantalla será la impresión de cada sensor con su respectiva lectura.

Flujo Principal: Coordinación de Sensores y Enum



2. Parte 2

2.1. Hito 1

1) Qué es lo que retorna la función clock().

R: clock retorna ciclos de reloj que se tienen que dividir entre CLOCKS_PER_SEC lo cual daría la cantidad de tiempo en segundos de cuanto tarda en ejecutarse el programa

2) Qué unidades tienen las variables inicio y fin.

R: Las variables inicio y fin tienen unidades en ciclos, o mejor llamado unidades de tiempo de reloj

3) Por qué para obtener el tiempo total de duración del ciclo for se debe dividir la diferencia entre fin e inicio con el valor de la constante `CLOCKS_PER_SEC`

R: Debido a que fin - inicio nos entrega ciclos de tiempo que tienen que ser divididos en la constante `CLOCKS_PER_SEC`

4) Explicar que hace (double) como prefijo de (fin - inicio):

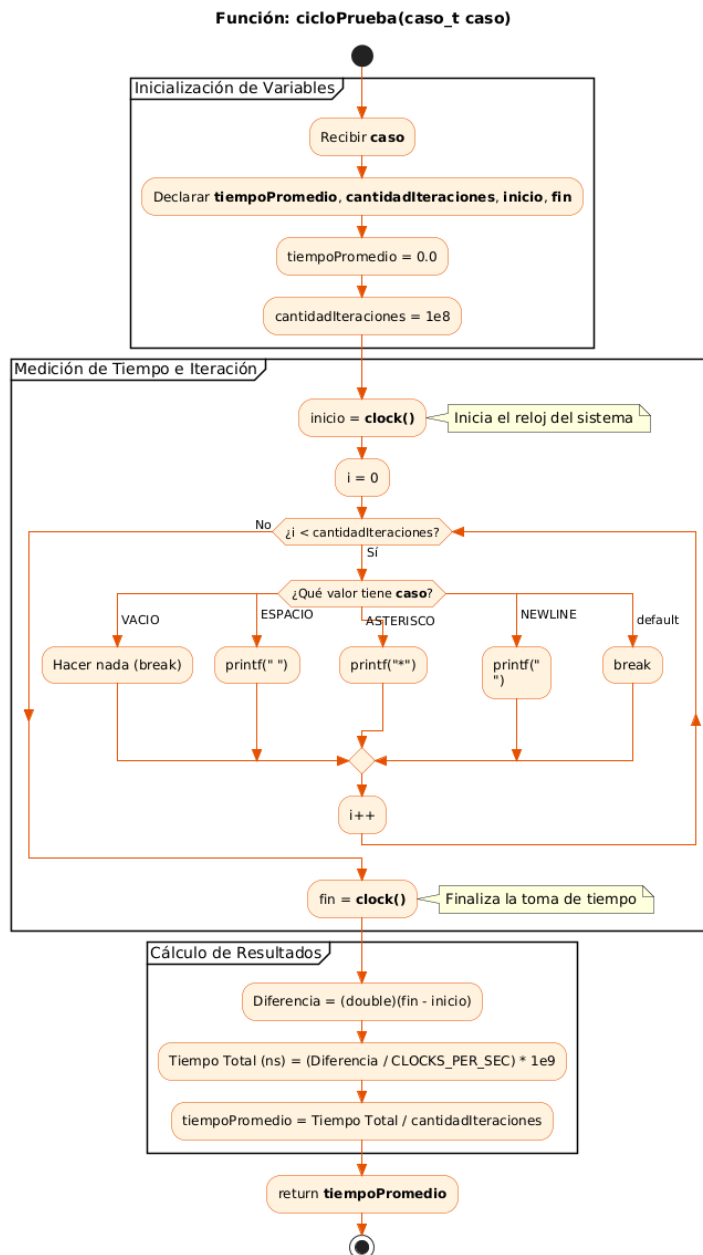
R: Es para expresar que el resultado será un double

5) Explicar para que sirve utilizar (double) como prefijo de (fin - inicio):

R: Sirve para hacer un cambio de variable a un flotante de doble precisión, cosa de no perder información de valores posiblemente decimales siendo enteros de 64 bits

2.2. Hito 2

`cicloPrueba()`, declarado como double, asigna a `caso_t` el caso que se le asignara al ser llamada la función, declara `tiempoPromedio`, `cantidadIteraciones`, `inicio` y `fin`, luego se le asigna un valor inicial a `tiempoPromedio` (0.0) y a `cantidadIteraciones` (1e8), después a `inicio` le asigna el valor de la función `clock()`, de esta forma se inicia el reloj, posteriormente, usando un `for()` se declara y asigna un valor inicial 0 a la variable `i`, se hace el caso en el que si `i` es menor a la cantidad de iteraciones se lleven a cabo el switch lleva a cabo la instrucción de imprimir vacío, espacios, asteriscos o líneas nuevas, dependiendo de los casos `VACIO`, `ESPACIO`, `ASTERISCO`, y `NEWLINE` del enum. Luego finaliza la toma de tiempo, por caso, asignando el valor que tenga `clock()` a la variable `fin`, finalmente se calcula el tiempo promedio haciendo la operación $((double)(fin - inicio) / CLOCKS_PER_SEC) * 1e9$, de esta forma tenemos la resta en una variable double para que en la siguiente línea obtener el `tiempoPromedio` que tardo en imprimir cada caso y retornar `tiempoPromedio`.



2.3. Hito 3

Estamos usando las variables inicio y fin las cuales están relacionadas con el inicio de la función clock() la cual devuelve el tiempo de proceso en ciclos que después es dividido por la constante CLOCKS_PER_SEC para que nos devuelva la cantidad de segundos que tardo el programa en ejecutarse. Como se mencionó en el Hito 2 esta estructurado de esa forma para poder tomar el tiempo de cada parte del enum y poder promediar cada caso

Para las pruebas bajamos las iteraciones a 1e6 debido a que con 1e8 tardaba demasiado en ejecutarse el programa. Además, se probó el programa 10 veces para dar aproximado de cuanto tarda el programa

Características del computador:

- 32 GB RAM
- AMD Ryzen 7 8700g w/ Radeon 780 m Graphics
- Almacenamiento 932 GB
- Windows 11

Versión computador: Versión 24H2 (compilación SO 26200.8328)

Versión compilador: g++ (tdm64-1) 10.3.0

Las mediciones descritas en la siguiente tabla representan el promedio de tiempo en nano-segundos que le tomo al programa procesar cada caso y cuánto tiempo le procesar los 4 casos en una iteración. Podemos ver que el proceso que menos tardo fue el VACIO, siendo casi 0 si no fuera que en una iteración dio 1 [ns], el proceso que más tardo en promedio fue el caso ESPACIO. Por último, en promedio una iteración tardo 109453,9000 [ns] en completarse, también podemos inferir que el programa completo tarda alrededor de 1 o 2 minutos en procesarse por completo.

	Muestra 1	Muestra 2	Muestra 3	Muestra 4	Muestra 5	Muestra 6	Muestra 7	Muestra 8	Muestra 9	Muestra 10
VACIO	0,0000	0,0000	0,0000	0,0000	0,0000	0,0000	0,0000	1,0000	0,0000	0,0000
ESPACIO	37043,0000	36990,0000	38831,0000	38959,0000	39034,0000	38340,0000	33607,0000	33700,0000	33896,0000	39772,0000
ASTERISCO	33993,0000	33338,0000	39034,0000	39423,0000	38962,0000	39535,0000	33675,0000	33826,0000	33710,0000	39658,0000
NEWLINE	37318,0000	33871,0000	39084,0000	38937,0000	36383,0000	34189,0000	33498,0000	33278,0000	33340,0000	39316,0000
TOTAL	108354,0000	104199,0000	116949,0000	117319,0000	114379,0000	112064,0000	100780,0000	100805,0000	100946,0000	118746,0000
	Promedio tiempo	desv. Estandar								
VACIO	0,1 [ns]	0,31622777 [ns]								
ESPACIO	37017,2 [ns]	2423,11181 [ns]								
ASTERISCO	36515,4 [ns]	2970,34679 [ns]								
NEWLINE	35921,4 [ns]	2571,47092 [ns]								
TOTAL	109454,1 [ns]	7353,73529 [ns]								

